Assignment 1

Due Date: 18th Feb 2026

Instructions:

- Assignments are to be done individually.
- No late assignments will be accepted.
- You must submit a single zip file containing your notebook or python files on Google Classroom named <your_student_id>.zip where <your_student_id> is something like i24-XXXX.
- Each file that you submit must contain your name, student-id, and assignment# on top of the file in comments.
- Follow the instructions. Assignments not following the instructions will be awarded zero points.
- Plagiarism: All submissions will be checked for plagiarism. Any evidence of plagiarism will result in a zero for the assignment and further disciplinary action as per the institution's policy.

Simple Reflex and Model Based Reflex Agents [40 Marks]

Q1. Consider you have an agent for a Jigsaw puzzle. The goal of the agent is to complete the Jigsaw puzzle in fixed number of moves. Your implementation should be modular so that the sensors, actuators, and environment characteristics (size etc.) can be changed easily. You need to implement two variants of intelligent agents and compare their performances.

1) Reflex agent
2) Reflex agent with state (model-based agent)

You can use the provided agents.py file (in resources) which contains the abstract implementation of agent and environment based on the course book. You can also make your own implementation but ensure proper documentation (explanation via comments).

Problem Description:

- Available actions are **Left**, **Right**, **Up**, **Down**, **NoOp**, **Rotate_left** and **Rotate_right**.
    - Rotate_left rotates the cell content by 90-degree counter clockwise
    - Rotate_right rotates the cell content by 90-degree clockwise
    - Left, Right, Up and Down move the cell content one step in that direction
    - NoOp means no movement will be made
- The termination criterion is that the agent runs out of moves or the puzzle is completed. Environment announces when this criteria is met and stops the execution.
- The performance of an agent is calculated after the termination criteria is met. The performance measure of an agent is the (# of correctly placed items) / (number of steps used). Note that there will be early termination if puzzle is solved before expiry of maximum number of moves.
- The environment is deterministic and partially observable.
- Agent knows the size of puzzle (grid n x n) and the content of the cell they land in, location of the landing cell (coordinates) is not known.
- The perception is given by the environment and includes, cell coordinates and if the current piece in the cell is rightly placed or not.
- Model based agent shall maintain the internal representation of the world and update it after receiving each perception. This state must be used to improve the performance of the agent and make the problem deterministic and fully observable.
- Starting position of the agent is random and not known beforehand plus puzzle contents are randomized at each start.
- Two agents must be placed the in the same randomized environment for fair comparison of performance
- Make sure to print the starting and final state of the puzzle.

Bonus:

- Visualize the puzzle solving process

Submission:

- You are required to submit a python file that is called in the following manner. First argument is puzzle size (50 x 50) and second argument is the maximum number of moves allowed (70).
- When the program is executed output should contain the following information:
  - Starting Puzzle State
  - Final Puzzle State
  - **`Reflex Agent:`** `No of correct pieces = 30, no of moves utilized = 150`
  - **`Model-based Agent:`** `No of correct pieces = 100, no of moves utilized = 100`

Search Algorithm Based Agent [60 Marks]

Q2. You are required to implement and compare multiple search algorithms (BFS, DFS, UCS, DLS, IDS, Greedy Best-First Search, and A*) on a dynamically changing maze. The goal is to develop algorithms that can efficiently navigate from a start position (S) to a goal position (G) under conditions where obstacles may move or appear/disappear during the search process.

1. **Maze Generation:**
   - **Grid Structure:**
     - The maze is to be represented as a grid of any size with same number of rows and columns but minimum size has to be more than 20 rows and 20 columns.
     - Each cell in the grid can either be empty (representing free space) or contain an obstacle.
   - **Initial Setup:**
     - The maze will be randomly generated with approximately 30% of the cells occupied by obstacles.
     - Ensure that there is a clear path from the start (S) to the goal (G) initially, meaning at least one valid path exists without obstacles blocking it.
   - **Positioning:**
     - The start (S) and goal (G) positions are randomly placed in empty cells within the maze.
     - Ensure that the start and goal are not too close to each other (at least

15 cells apart).

- **Maze Representation:**
  - Use a 2D list or numpy array to represent the maze, where each cell can be:
    - `0` for an empty cell,
    - `1` for an obstacle,
    - `S` for the start position,
    - `G` for the goal position.

2. **Dynamic Obstacles:**
   - **Obstacle Behavior:**
     - Every 10 moves made by the search algorithm, introduce a new obstacle at a random empty cell. This obstacle will remain in place for the remainder of the search.
     - Every 15 moves, select an existing obstacle (if any) to move to an adjacent empty cell (up, down, left, or right). If all adjacent cells are occupied or out of bounds, the obstacle does not move.
   - **Implementation Requirements:**
     - Implement functions to handle the addition and movement of obstacles. These functions should be called during the search process based on the move count.
     - If an obstacle blocks the path between the start and the current position of the algorithm, the algorithm must detect this and reroute or adapt as needed.

3. **Search Algorithms Implementation:**
   - Implement the following search algorithms in Python:
     1. **Breadth-First Search (BFS):**
        - Ensure that the algorithm handles cycles or repeated nodes.
        - **Note:** BFS does not involve backtracking; if a path is blocked, the algorithm will explore alternative paths in the breadth-first manner.
     2. **Depth-First Search (DFS):**
        - Backtracking will be necessary if the algorithm reaches a dead end.
     3. **Uniform Cost Search (UCS):**
        - Cost per Move: Assume that each move has a cost of 1.
     4. **Depth-Limited Search (DLS):**

        - Set the depth limit as an input parameter and test with various limits.
        - Backtracking will be used if the depth limit is reached without finding the goal.
    5. **Iterative Deepening Search (IDS):**
        - Ensure the algorithm correctly resets and starts afresh with each new depth limit.
        - Backtracking will be necessary as in DFS.
    6. **Greedy Best-First Search:**
        - Use a priority queue where nodes are prioritized based on their heuristic value (Manhattan distance to the goal).
    7. ***A* Search:***
        - Implement A* using a priority queue where the priority is the sum of the path cost so far and the heuristic (Manhattan distance).
        - The cost to move from one node to its neighbor is typically uniform (e.g., cost per move = 1). This means g(n) is simply the number of moves made from the start to node n.
        - Backtracking will be necessary if the algorithm finds a lower-cost path to a previously explored node.

4. **Algorithm Adaptation:**
    - **Dynamic Handling:**
        - The search algorithms must be capable of adapting to changes in the maze during execution. For example, if an obstacle appears in the current path, the algorithm should detect this and find an alternative route.
        - Implement a detection mechanism that checks if a newly introduced or moved obstacle blocks the planned path. If blocked, the algorithm must reroute or backtrack (if applicable) as needed.
    - **Continuous Operation:**
        - The search should not restart entirely unless necessary. Instead, it should continue from the current position and adapt to the new maze configuration.
        - Consider implementing a mechanism to store and update paths as obstacles are added or moved.

5. *Heuristic for Greedy and A:**
    - **Manhattan Distance:**
        - Use the Manhattan distance (sum of the absolute differences between

the x and y coordinates) as the heuristic for both Greedy Best-First Search and A* Search.

- **Heuristic Function Implementation:**
  - Write a separate heuristic function that calculates the Manhattan distance from any given node to the goal. This function will be used by both Greedy and A* Search algorithms.

    If you are at position (2, 3) and the goal is at (5, 7) on a grid:

    Calculating the Manhattan Distance:

    $\text{Distance} = |5-2| + |7-3| = 3+4 = 7$

    This value (7) will be used as h(n) in the A* algorithm for this particular node.

6. **Performance Evaluation:**
   - **Metrics to Measure:**
     - **Time Complexity:** Measure the time taken by each algorithm to find the goal. Include time to adapt to new obstacles.
     - **Space Complexity:** Track the number of nodes expanded during the search, including those revisited due to dynamic changes.
     - **Memory Usage:** Monitor the memory footprint of each algorithm during execution.
     - **Success Rate:** Determine how often each algorithm successfully finds the goal despite the dynamic changes in the maze.
   - **Evaluation Procedure:**
     - Run each algorithm on multiple randomly generated mazes with dynamic obstacles and record the performance metrics.
     - Compare the algorithms based on the recorded data and analyze their strengths and weaknesses.

7. **Output Requirements:**
   - **Path Representation:**
     - For each algorithm, print the final path from start (S) to goal (G), if found, as a list of grid coordinates (e.g., [(0,0), (0,1), ..., (19,19)]).
     - Indicate any points where the algorithm had to reroute due to dynamic obstacles.
   - **Move Count:**
     - Display the number of moves taken to reach the goal.

        - Include a breakdown of how many moves were spent backtracking or rerouting (if applicable).
    - **Nodes Expanded:**
        - Print the total number of nodes expanded during the search.
        - Highlight any redundant expansions caused by dynamic changes.
    - **Maze Visualization:**
        - Provide a step-by-step visualization of the maze, showing the algorithm's progress, obstacles appearing or moving, and the path taken.
        - Consider using matplotlib or a similar library to create this visualization.

8. **Bonus:**
    - **Moving Start/Goal Feature:**
        - Implement a feature where either the start (S) or goal (G) position moves to an adjacent cell every 20 moves. The search algorithm must continue from the new start/goal position.
        - The movement of S or G should be random but within the maze bounds and not into an obstacle.
        - Adapt the search algorithm to handle this additional dynamic challenge and still reach the goal.

Grading Criteria:

- **Correctness:** The algorithms should correctly find the path from start to goal, adapting to dynamic changes.
- **Completeness:** Ensure all algorithms are fully implemented, including handling of dynamic obstacles and additional features.
- **Efficiency:** Evaluate how efficiently the algorithms run, considering both time and space complexity.
- **Clarity:** Code should be clear, well-organized, and easy to understand. Include comments and documentation.
- **Adaptation:** Algorithms should demonstrate robust adaptation to dynamic changes in the maze.
- **Performance Comparison:** Include a section in your submission that compares the performance of each algorithm. This should include tables, charts, or graphs to visualize the data.